

PLANIFICACIÓN · V2.0 · JUNIO 2026

Elfie

PRTS Desktop Edition

Evolución de PRTS a aplicación de escritorio nativa con control del sistema operativo, IA local, voz configurable y personalidad propia. Base de datos compartida con la versión web. Uso exclusivo de Sylft.

PROPIETARIO: SYLFT

IA: ELFIE

FRAMEWORK: TAURI 2.x

RTX 5060 · 8 GB VRAM

16 GB RAM · RYZEN 5 5600G

WINDOWS 10 / 11

MONOREPO: sylftkto/mi-app-PRTS · BRANCH: elfie-desktop/ · BD: SUPABASE COMPARTIDA

Contenido

1 · Visión y Diferencias vs PRTS Web	3
2 · Estructura del Repositorio (Monorepo)	4
3 · Compatibilidad Windows 10	4
4 · Arquitectura del Sistema	5
Hardware y presupuesto de VRAM	5
Diagrama de sistema	5
5 · Módulos del Sistema	6
Control del SO · Monitor · Personalidad Elfie	6
6 · Capa de Voz (STT + TTS)	7
Whisper.cpp · Kokoro · XTTS v2 · Opciones cloud	7
7 · IA Local con Ollama (RTX 5060)	8
Modelos · Router híbrido · Ahorro estimado	8
8 · Base de Datos Compartida	9
Nuevas tablas · Sincronización Realtime	9
9 · Fases de Desarrollo (Reestructuradas)	10
10 · Mejoras Técnicas Internas	11
RAG local (LanceDB) · Global Shortcuts	11
11 · Expansiones de Capacidad	12
Routine Engine Visual · Integración Obsidian	12
12 · Integraciones y Visión a Largo Plazo	13

Visión y Diferencias vs PRTS Web

PROPÓSITO

Elfie-PRTS Desktop extiende PRTS web sin reemplazarla. Mientras PRTS web es accesible desde cualquier dispositivo, Elfie vive permanentemente en la máquina de Sylft con acceso nativo al SO, IA local y personalidad configurable. Comparten la misma base de datos Supabase — los datos siempre están sincronizados.

Qué cambia y añade Elfie Desktop

CAPACIDAD	PRTS WEB (ACTUAL)	ELFIE DESKTOP (NUEVO)
Control del SO	Solo protocol handlers (discord://, spotify:...)	Nativo: procesos, ventanas, volumen, archivos
STT	webkitSpeechRecognition → Google Cloud	Whisper.cpp local, sin Google, funciona offline
Wake word	webkitSpeechRecognition continuo (Dalia)	Picovoice Porcupine on-device (real nivel SO)
TTS / Voz de Elfie	speechSynthesis del navegador	Kokoro / Piper local · XTTS v2 (clonación)
LLM	Solo Anthropic API (costo por llamada)	Ollama local (RTX 5060) + Anthropic fallback
Accesos rápidos	Sin atajos globales	Ctrl+Space · Alt+E desde cualquier app
Búsqueda en notas	Búsqueda textual simple	RAG local semántico (Apuntes)
Rutinas de inicio	Hardcodeadas en actions.js	Routine Engine Visual — configurables en UI
Monitoreo	Sin métricas del sistema	CPU / GPU / RAM / temperatura en tiempo real
Disponibilidad	Necesita navegador abierto	Tray icon · siempre activo · auto-start

Estado actual de PRTS Web — Base que se reutiliza

MÓDULO	ESTADO	EN ELFIE DESKTOP
Dashboard · Tareas · Semana · Proyectos · Insights	ACTIVO	Reutilizado en WebView sin cambios
Gym · Dieta · Apuntes · Finanzas · Recordatorios	ACTIVO	Reutilizado en WebView sin cambios
Capa IA: 4 Edge Functions + circuit breaker	ACTIVO	Reutilizada · Ollama la sustituye parcialmente
Wake word "Dalia" · TTS · Voz push-to-talk	ACTIVO	Reemplazado por Whisper + Kokoro + Porcupine
Alumnos · Contenido	PENDIENTE	Se implementan en Fase 5 del desktop

Estructura del Repositorio

(Monorepo)

02

DECISIÓN

Mismo repositorio — carpeta elfie-desktop/ como módulo nuevo

Tauri carga `../app` directamente como su WebView. Separar repos implicaría mantener dos copias del mismo frontend HTML/JS en sincronía. Las migraciones de Supabase son compartidas. Para un solo desarrollador/usuario es la arquitectura correcta.

```
mi-app-PRTS/ |─ app/ ← frontend compartido (HTML/JS vanilla – sin cambios) | |─
index.html ← dashboard · tareas · semana · proyectos · insights | |─ gym.html ·
dieta.html · apuntes.html · finanzas.html · recordatorios.html | |─ config.js ←
SUPABASE_URL · SUPABASE_ANON_KEY · GOOGLE_CLIENT_ID | |─ styles.css · sw.js ·
manifest.json | |─ ai/ | |─ actions.js · voice.js · wakeword.js · gcal.js | |─
supabase/ ← BD compartida (sin cambios) | |─ migrations/ ← 0001–0011 existentes ·
0012–0015 nuevas (elfie) | |─ functions/ ← Edge Functions existentes reutilizadas | |─
_shared/ · generate-briefing/ · process-inbox/ · ... | |─ elfie-desktop/ ← NUEVO: app
Tauri 2.x | |─ src-tauri/ ← Rust backend | | |─ Cargo.toml | | |─ tauri.conf.json ←
distDir: "../..../app" (apunta al frontend compartido) | | |─ src/ | | |─ main.rs | | |─
system_control.rs ← open_app · volumen · archivos · portapapeles | | |─ monitor.rs ← CPU/
RAM (sysinfo) · GPU (nvmf-wrapper) | | |─ audio.rs ← Porcupine wake word · Whisper STT ·
TTS engines | | |─ ai_router.rs ← Ollama local · Anthropic fallback | | |─ package.json ←
solo Tauri CLI ("@tauri-apps/cli") | | |─ docs/ | | |─
Elfie_PRTS_Desktop_Planificacion_v2.0.pdf ← este documento | | |─ Fase4_Arquitectura_IA.md
· Fase5_WakeWord... | | |─ CLAUDE.md · PRODUCT.md · DESIGN.md | | |─ package.json ← npm run
dev (serve app/) · db:push · db:new
```

TAURI APUNTANDO AL FRONTEND DE PRTS

En `elfie-desktop/src-tauri/tauri.conf.json`, la clave `"distDir": "../..../app"` hace que el WebView cargue la misma carpeta que ya sirve Vercel. Cero duplicación de código frontend.

Compatibilidad Windows 10

CONTEXTO IMPORTANTE

Windows 10 llegó a su fin de soporte el **14 de octubre de 2025**. A junio de 2026 ya no recibe parches de seguridad. Elfie Desktop puede desarrollarse y correrse en Windows 10, pero hay consideraciones que vale tener presentes.

Qué funciona igual en Windows 10

Tauri 2.x + WebView2

Compatible. En Windows 10 WebView2 no siempre está preinstalado, pero Tauri puede empaquetarlo automáticamente con `webview2InstallMode = "embedBootstrapper"` en `tauri.conf.json`. Transparente para el usuario.

Ollama + Whisper + Kokoro

Todos soportan Windows 10 oficialmente. No dependen de APIs del SO — corren sobre CUDA y DirectX disponibles en Win10.

Auto-start · Tray · Shortcuts

Registro de Windows (auto-start), tray icon y global shortcuts funcionan exactamente igual en Windows 10 y 11.

Notificaciones nativas

Toast Notifications disponibles desde Windows 10 1809+. Tauri las usa por defecto sin configuración adicional.

Rutas y LAUNCHERS existentes

Las rutas en `actions.js` (`C:/Users/Sylft/...`) ya son rutas de Windows. Sin cambio al migrar el launcher a Tauri.

Picovoice Porcupine

SDK disponible para Windows 10. Requiere Visual C++ Redistributable (normalmente ya instalado).

Puntos a verificar antes de comprometerse

Drivers RTX 5060 en Win10

La RTX 5060 (Blackwell, 2025) es posterior al EOL de Win10. NVIDIA generalmente mantiene soporte de drivers, pero verificar que los drivers actuales de la tarjeta soporten CUDA en Windows 10 antes de asumir que Ollama/Whisper funcionan con GPU.

Seguridad — SO sin parches

Elfie Desktop maneja tokens de Supabase y OAuth de Google. Sin actualizaciones de seguridad del SO desde octubre 2025, existe riesgo real. Recomendable migrar a Windows 11 antes o durante Fase 4 (IA local).

RECOMENDACIÓN

Desarrollar normalmente en Windows 10. El código no cambia entre versiones de Windows. Verificar drivers de la RTX 5060 antes de la Fase 3 (voz local). Planificar

migración a Windows 11 antes de la Fase 4 para aprovechar mejor el hardware y resolver el riesgo de seguridad.

Arquitectura del Sistema

Hardware disponible y presupuesto de VRAM

GPU

RTX 5060 · 8 GB

Whisper: ~1.5 GB (siempre)
LLM Mistral 7B Q4: ~4.5 GB (al procesar)
XTTS v2: ~2.5 GB (al sintetizar, ~2 s)
Peak máximo: ~7.5 GB

RAM

16 GB DDR4

Tauri + frontend: ~200 MB
Ollama runtime: ~500 MB
Sistema + otras apps: ~8 GB
Disponible para Elfie: ~7 GB

CPU

Ryzen 5 5600G · 6C/12T

Rust backend (Tauri): mínimo
Porcupine wake word: ~3% CPU
Piper TTS (fallback CPU): <50 ms
LanceDB RAG queries: <100 ms

Diagrama de sistema

```
┌─── ELFIE DESKTOP (Tauri 2.x) ───┐ | | |
Frontend (WebView – app/ compartido) Core Rust (Tauri IPC) | |
┌──────────────────────────────────┐ | | Módulos PRTS
(sin cambios) | | system_control.rs | | | + app/elfie/ |◀──▶| open_app · volumen | | |
| personality.js | IPC | archivos · portapapeles | | | voice_desktop.js | | | |
routines.js (nuevo) | | monitor.rs | | | rag.js (nuevo) | | sysinfo (CPU/RAM) | | |
┌──────────────────────────────────┐ | nvml-wrapper (RTX 5060) | | | | | Atajos
Globales (tauri-plugin-global-shortcut) audio.rs | | | Ctrl+Space → abrir captura |
Porcupine (wake word) | | | Alt+E → activar escucha | Whisper.cpp (STT) | | | Kokoro /
Piper (TTS) | | | RAG local (LanceDB) | | | LanceDB en disco (~elfie-desktop/db/) |
ai_router.rs | | | Embeddings de Apuntes | Ollama (local) ↔ | | | Consultas semánticas
por voz/texto | Anthropic (fallback) | | | ┌──────────────────┐ |
┌──────────────────────────────────┐ |
┌──────────────────────────────────┐ | SUPABASE (compartido con PRTS
web) | | Tablas existentes (RLS) · Realtime websocket | | Nuevas: elfie_config ·
voice_profiles | | system_actions · system_metrics · routines |
┌──────────────────────────────────┐ | PRTS Web (Vercel) – misma BD ·
sync <1 s bidireccional
```


Módulos del Sistema

Control del Sistema Operativo (Rust)

COMANDO TAURI IPC	FUNCIÓN	ACTIVACIÓN
<code>open_app</code>	Lanza ejecutables por nombre; alias configurables en <code>elfie_config</code>	"Elfie, abre Discord"
<code>kill_process</code>	Termina procesos por nombre o PID	"Elfie, cierra Steam"
<code>run_routine</code>	Ejecuta secuencia del Routine Engine	"Elfie, modo estudio"
<code>set_volume</code>	Volumen maestro del sistema (0–100)	"Elfie, sube el volumen a 60"
<code>mute_toggle</code>	Silenciar/activar audio	"Elfie, silencia"
<code>clipboard_read/ write</code>	Leer o escribir el portapapeles	"Elfie, ¿qué tengo copiado?"
<code>open_folder</code>	Abre carpeta en Explorador de Windows	"Elfie, abre mis proyectos"
<code>screenshot</code>	Captura de pantalla → disco o clipboard	"Elfie, toma una captura"
<code>power_action</code>	Apagar / reiniciar / suspender	"Elfie, suspende la computadora"

Monitor de Recursos (crates: sysinfo + nvml-wrapper)

Emite datos cada 3 s al frontend vía eventos Tauri. El panel vive en el dashboard de Elfie y puede mostrarse como widget flotante.

MÉTRICAS DISPONIBLES

- CPU (Ryzen 5 5600G): uso total y por núcleo · frecuencia actual · temperatura
- GPU (RTX 5060): % uso · VRAM usada/total (8 GB) · temperatura · potencia · ventilador
- RAM: usada / disponible / total (16 GB)
- Disco: espacio libre por partición
- Red: Mbps bajada y subida en tiempo real
- Procesos: top-10 por CPU y por RAM

Personalidad de Elfie

PERFIL	TONO	TTS	VERBOSIDAD
Profesional (default)	Directa, eficiente, sin rodeos	Voz neutra · velocidad 1.0	Concisa
Casual	Informal, comentarios breves	Voz cálida · velocidad 1.05	Media
Silenciosa	Solo confirmaciones de 1–2 palabras	TTS desactivado · solo texto	Mínima
Técnica	Datos concretos, sin suavizar	Voz neutra · velocidad 0.95	Detallada
Nocturna	Suave, volumen reducido automático	Voz suave · velocidad 0.9	Mínima

Persiste en `elfie_config.personality` (Supabase) — sincronizada entre desktop y web.

STT — Whisper.cpp (reemplaza webkitSpeechRecognition)

CARACTERÍSTICA	WEBKITSPEECHRECOGNITION (ACTUAL)	WHISPER.CPP (DESKTOP)
Privacidad	Audio enviado a Google	100% local, sin Google
Offline	Requiere internet	Funciona sin internet
Precisión es-MX	Buena	Superior en español
VRAM requerida	0 (cloud)	~1.5 GB (large-v3-turbo)
Latencia	~500 ms	~300 ms (frase corta, GPU)

TTS — Motores Locales (sin costo)

Kokoro TTS PRIMARIO LOCAL · 82M PARAMS · ONNX · GPU Calidad Excelente Latencia <100 ms (GPU) VRAM ~0.5 GB Clonación No	Piper TTS FALLBACK CPU LOCAL · CPU · ULTRARRÁPIDO Calidad Buena Latencia <50 ms (CPU) VRAM 0 — solo CPU Uso Si GPU ocupada
XTTS v2 (Coqui) CLONACIÓN LOCAL · CLONA VOZ CON 5 S DE AUDIO Calidad Muy alta Latencia ~500 ms (GPU) VRAM ~2.5 GB Clonación Sí — cualquier voz	Fish Speech ALTERNATIVA LOCAL · ESTADO DEL ARTE Calidad Estado del arte Latencia ~200 ms (GPU) VRAM ~4 GB Clonación Sí

TTS — Fallback Cloud

edge-tts: es-MX-DaliaNeural GRATUITO MICROSOFT EDGE TTS · SIN API KEY Costo Gratis (500k chars/mes) Latencia ~200 ms Nota La voz se llama "Dalia" — misma wake word	ElevenLabs CLOUD · MÁXIMA CALIDAD · CLONACIÓN Costo \$5-22/mes Calidad La mejor del mercado Uso Opcional — momentos especiales
--	---

STACK DE VOZ — ESTRATEGIA EN CAPAS

- 1 · **Kokoro (local GPU)** — respuestas cotidianas · <100 ms · \$0
- 2 · **Piper (local CPU)** — fallback si GPU saturada · <50 ms · \$0
- 3 · **edge-tts es-MX-DaliaNeural (cloud gratuita)** — si modelos locales no están cargados
- 4 · **XTTS v2 (local)** — perfil de voz clonada para identidad de Elfie · Fase 6

IA Local con Ollama (RTX 5060)

Modelos recomendados para 8 GB VRAM

Phi-3.5 Mini · Q4_K_M

VRAM ~2.5 GB

Velocidad ~120 tok/s

Calidad Muy buena

Uso ideal Router · comandos rápidos · respuestas simples

Qwen2.5 7B · Q4_K_M

VRAM ~4.5 GB

Velocidad ~50 tok/s

Calidad Excelente en español

Uso ideal Inbox · clasificación · razonamiento

Llama 3.1 8B · Q4_K_M

VRAM ~5.0 GB

Velocidad ~40 tok/s

Calidad Excelente

Uso ideal Conversación profunda · intent ask

Mistral 7B v0.3 · Q4_K_M

VRAM ~4.5 GB

Velocidad ~50 tok/s

Calidad Excelente

Uso ideal Clasificación de texto · análisis

Router Híbrido — Local + Nube

Elfie recibe comando (voz o texto) | ▼ ¿Patrón local sin LLM? —Sí→ Resolver en cliente (0 ms · \$0) | NO hora · fecha · saludo · navegar · abrir apps · clima ▼ ¿Ollama disponible? —Sí→ Phi-3.5 Mini / Qwen2.5 7B local (~300-800 ms · \$0) | NO inbox · clasificación · preguntas · comandos complejos ▼ ¿Anthropic API disponible? —Sí→ Claude Haiku 4.5 (~600 ms · ~\$0.001/llamada) | NO briefing diario · insights (calidad máxima) ▼ Modo degradado — respuesta determinista / manual (sin IA)

AHORRO ESTIMADO CON IA LOCAL

PRTS web gasta Anthropic API en: clasificación inbox (~5 llamadas/día) + comandos de voz (~10/día) + briefing (1/día). Con Ollama local, las clasificaciones y comandos simples cuestan \$0. **Reducción estimada: 70–80% del gasto mensual.** Solo briefings e insights quedan en Claude API por calidad de contexto amplio.

08

Base de Datos Compartida

La base de datos Supabase existente es la fuente de verdad única. Elfie Desktop usa las mismas credenciales, el mismo RLS y el mismo cliente JavaScript de PRTS. Se añaden 5 migraciones nuevas.

Nuevas Tablas (migraciones 0012–0016)

```
-- 0012: Configuración de Elfie (personalidad, voz, modelo local) CREATE TABLE elfie_config
( user_id uuid DEFAULT auth.uid() PRIMARY KEY, personality text DEFAULT 'profesional',
voice_engine text DEFAULT 'kokoro', -- kokoro|piper|xtts|edge|elevenlabs voice_name text
DEFAULT 'es-MX-DaliaNeural', voice_speed float DEFAULT 1.0, verbosity text DEFAULT 'concisa',
tts_enabled bool DEFAULT true, local_llm_model text DEFAULT 'phi3.5', updated_at timestampz
DEFAULT now() ); -- 0013: Perfiles de voz clonados (XTTS v2 / Fish Speech) CREATE TABLE
voice_profiles ( id uuid DEFAULT gen_random_uuid() PRIMARY KEY, user_id uuid DEFAULT
auth.uid(), name text NOT NULL, engine text NOT NULL, reference_audio_url text, config jsonb,
is_active bool DEFAULT false, created_at timestampz DEFAULT now() ); -- 0014: Log de
acciones del sistema operativo CREATE TABLE system_actions ( id uuid DEFAULT
gen_random_uuid() PRIMARY KEY, user_id uuid DEFAULT auth.uid(), action_type text NOT NULL,
payload jsonb, executed_at timestampz DEFAULT now() ); -- 0015: Métricas del sistema
(historial para gráficas) CREATE TABLE system_metrics ( id uuid DEFAULT gen_random_uuid()
PRIMARY KEY, user_id uuid DEFAULT auth.uid(), cpu_percent float, ram_used_gb float,
gpu_percent float, gpu_vram_used_gb float, gpu_temp_c float, gpu_power_w float, recorded_at
timestampz DEFAULT now() ); -- 0016: Rutinas configurables (Routine Engine Visual) CREATE
TABLE routines ( id uuid DEFAULT gen_random_uuid() PRIMARY KEY, user_id uuid DEFAULT
auth.uid(), name text NOT NULL, trigger_phrase text, -- "modo estudio" shortcut text, --
"Ctrl+Shift+E" steps jsonb NOT NULL, -- [{action, params, delay_ms}] is_active bool DEFAULT
true, created_at timestampz DEFAULT now() );
```

Sincronización en Tiempo Real (Supabase Realtime)

CANAL	DIRECCIÓN	EFFECTO EN ELFIE
tasks	Web ↔ Desktop	Tarea creada en web → aparece en Elfie en <1 s
daily_briefings	Web → Desktop	Briefing pre-generado → notificación nativa
reminders	Web ↔ Desktop	Recordatorio próximo → alerta de sistema
inbox_entries	Web ↔ Desktop	Capturas procesadas en cualquier cliente
routines	Desktop → Web	Nueva rutina creada en desktop (solo aplica en desktop)

Fases de Desarrollo (Restructuradas)

09

F0 Setup Base

~1 sem

- Crear `elfie-desktop/` en el repo
- Inicializar Tauri 2.x
- `distDir: "../../app"` → PRTS en WebView
- WebView2 embebido (Windows 10 compat.)
- Conexión Supabase reutilizada
- Verificar que los 10 módulos corren en WebView

F1 Core Desktop

~2 sem

- Tray icon + menú contextual
- Auto-start en Windows (registro)
- Notificaciones nativas de Windows
- Supabase Realtime (sync live)
- **tauri-plugin-global-shortcut**
INFRAESTRUCTURA
- Ctrl+Space = captura · Alt+E = voz

F2 Control del Sistema

~2 sem

- Launcher de aplicaciones (`open_app`)
- Control de volumen nativo
- Monitor CPU/GPU/RAM (NVML)
- Gestión de archivos y carpetas
- Portapapeles (leer/escribir)
- Screenshot nativo

F3 Voz Local

~2 sem

- Whisper.cpp STT (sin Google)
- Kokoro TTS + Piper fallback
- Picovoice Porcupine wake word
- Flujo: wake → Whisper → router → TTS
- Indicador visual en tray (escucha/habla)
- Verificar drivers RTX 5060 en Windows

F4 IA Local

~2-3 sem

- Instalar Ollama + modelos base
- Router híbrido local → Anthropic
- Personalidad Elfie configurable
- Panel de personalidad en la UI
- Perfiles de voz (múltiples motores)
- *Migración recomendada a Win11 aquí*

F5 Módulos Nuevos

~2-3 sem

- **Routine Engine Visual** **ALTA PRIORIDAD**
- Panel de monitor de sistema completo
- Historial de métricas en gráficas
- Módulo Alumnos (tabla + UI)
- Módulo Contenido (planificador)

F6 RAG + Obsidian + Voz

~2 sem

- **RAG local (LanceDB)** para Apuntes
semántico
- Embeddings de notas al crear/editar
- "Elfie, ¿qué anotamos sobre X?"
- XTTS v2 clonación de voz

F7 Distribución

~1 sem

- Installer .msi / .exe (WebView2 embebido)
- Script de setup (Ollama + modelos)
- Documentación de configuración inicial
- Firma del ejecutable (opcional)

- Obsidian export (*si lo usás activamente*)

TIEMPO TOTAL ESTIMADO: ~13-15 SEMANAS

Fases 0–3 entregan la app funcional básica con voz local en ~7 semanas. Fases 4–7 añaden IA local, Routine Engine, RAG y distribución. Misma filosofía de PRTS: uso real después de cada fase antes de avanzar.

Mejoras Técnicas Internas

RAG Local con LanceDB

QUÉ RESUELVE

El módulo Apuntes de PRTS tiene notas por materia con 5 campos (conceptos, fórmulas, dudas, conexiones, resumen). Hoy solo se puede buscar por texto exacto. Con RAG, Elfie puede responder preguntas semánticas: "¿Qué anotamos sobre complejidad algorítmica?" aunque no uses esas palabras exactas.

ALCANCE CLARO

Para qué sirve el RAG en PRTS: exclusivamente para el módulo **Apuntes** — el resto de los módulos ya son datos estructurados en Supabase que el LLM puede consultar directamente con SQL. RAG no aporta nada que una query SQL no haga mejor para tareas, gym o finanzas.

IMPLEMENTACIÓN

```
// Flujo RAG para Apuntes (LanceDB local) // 1. Al crear/editar una nota → generar embedding
const embedding = await ollama.embeddings({ model: 'nomic-embed-text', // modelo de
embeddings, ~270MB VRAM prompt: nota.conceptos + ' ' + nota.resumen }); // 2. Guardar en
LanceDB (elfie-desktop/db/apuntes.lance) await table.add([{ id: nota.id, materia:
nota.materia, vector: embedding.embedding, text: nota.resumen }]); // 3. Al preguntar por voz
→ buscar similares → prompt al LLM const resultados = await
table.search(queryEmbedding).limit(3).execute(); const contexto = resultados.map(r =>
r.text).join(' '); // → LLM recibe contexto + pregunta → responde basado en TUS notas
```

LanceDB es un vector store embebido (como SQLite pero para vectores). No requiere servidor ni Docker. Los embeddings se generan con Ollama local (`nomic-embed-text` , ~270 MB VRAM) y viven en disco. **Privacidad total: tus notas nunca salen de la computadora.**

CUÁNDO IMPLEMENTARLO

RAG tiene sentido cuando hay suficiente volumen de notas (20+ apuntes distintos). Implementarlo en Fase 6 da tiempo para acumular contenido real y validar que el caso de uso efectivamente ocurre en el uso diario.

tauri-plugin-global-shortcut

FASE 1 – INFRAESTRUCTURA

POR QUÉ VA EN FASE 1, NO EN FASE 5

Sin atajos globales, para interactuar con Elfie tenés que cambiar de ventana y hacer clic en la app. Eso destruye el propósito de un asistente de escritorio. Los atajos globales son la diferencia entre un asistente que interrumpe el flujo y uno que lo complementa.

```
// src-tauri/src/main.rs use tauri_plugin_global_shortcut::{Code, Modifiers, ShortcutState};
app.plugin(tauri_plugin_global_shortcut::Builder::new().build())?; // Registrar al iniciar la
app let app_handle = app.handle().clone(); app.global_shortcut().on_shortcuts( ["Ctrl+Space",
"Alt+E"], move |_app, shortcut, event| { if event.state() == ShortcutState::Pressed { match
shortcut.key_string() { "ctrl+space" => app_handle.emit("elfie:open-capture", ()), "alt+e" =>
app_handle.emit("elfie:start-voice", ()), _ => {} } } } )?;
```

ATAJO	ACCIÓN	FUNCIONA DESDE
Ctrl+Space	Abrir input de captura de Elfie	Cualquier app activa
Alt+E	Activar escucha de voz (push-to-talk)	Cualquier app activa
Ctrl+Shift+R	Ejecutar rutina rápida (configurable)	Cualquier app activa
Personalizable	Cualquier acción de Elfie	Cualquier app activa

Expansiones de Capacidad

Routine Engine Visual

ALTA PRIORIDAD – Fase 5

QUÉ RESUELVE

Las rutinas de inicio en PRTS web están hardcodeadas en `actions.js`: "Buen día" abre *Discord* + *Spotify* + *YouTube* escalonados. Para cambiar qué abre, hay que editar el archivo. El Routine Engine Visual convierte esas rutinas en datos configurables desde la UI, sin tocar código.

INTERFAZ DE CREACIÓN DE RUTINAS

Panel "Mis Rutinas" en Elfie Desktop [+ Nueva rutina] Nombre: "Modo estudio" Activar con voz: "modo estudio" Atajo de teclado: `Ctrl+Shift+E` Pasos: 1. Abrir VS Code [delay: 500ms] 2. Abrir Spotify → playlist foco [delay: 800ms] 3. Silenciar Discord [delay: 300ms] 4. Volumen del sistema → 40% [delay: 0ms] 5. Abrir PRTS → vista Tareas [delay: 600ms] [Probar rutina] [Guardar] [Eliminar] — Rutinas guardadas: • Modo estudio `Ctrl+Shift+E` · "modo estudio" • Modo entrenamiento `Ctrl+Shift+G` · "modo gym" • Modo clase Wolves `Ctrl+Shift+W` · "modo clase" • Buen día — · "buen día" / "buenos días"

MODELO DE DATOS

```
-- Tabla routines (migración 0016) steps: jsonb = [ { "action": "open_app", "params": { "name": "vscode" }, "delay_ms": 500 }, { "action": "spotify_preset", "params": { "preset": "foco" }, "delay_ms": 800 }, { "action": "set_volume", "params": { "level": 40 }, "delay_ms": 300 }, { "action": "navigate", "params": { "view": "tareas" }, "delay_ms": 600 } ]
```

El ejecutor de rutinas es una extensión del `aplicarAccion()` existente en `actions.js` — ya sabe abrir apps, navegar vistas y controlar Spotify. Solo se añade la capa de secuenciación con delays.

Integración con Obsidian

CONDICIONAL – solo si usás Obsidian

EVALUACIÓN HONESTA

Esta integración solo tiene valor si Obsidian ya es parte de tu flujo diario de notas. Si no lo usás, implementarla solo crea dependencia sin beneficio. Evaluá primero si el RAG local (Fase 6) resuelve la búsqueda semántica que necesitás — probablemente sí.

SI SE IMPLEMENTA: SINCRONIZACIÓN PRTS → OBSIDIAN

DIRECCIÓN

COMPLEJIDAD

CÓMO

BAJA

DIRECCIÓN	COMPLEJIDAD	CÓMO
PRTS → Obsidian (export)		Tauri escribe archivos .md al vault de Obsidian cuando se crea/edita una nota en PRTS. Formato: frontmatter YAML + contenido.
Obsidian → PRTS (sync bidireccional)	ALTA	Requiere watcher de archivos (<code>tauri-plugin-fs-watch</code>) + parseo de Markdown. Conflictos posibles si se edita en ambos lados. No recomendado.

```
// Export de nota a .md (Tauri Rust → disco) fn export_to_obsidian(nota: &Nota, vault_path:
&str) { let md = format!("--- id: {} materia: {} fecha: {} --- ## Conceptos {} ## Resumen
{}", nota.id, nota.materia, nota.created_at, nota.conceptos, nota.resumen);
std::fs::write(format!("{}/{}/*.md", vault_path, nota.materia, nota.title), md); }
```

Integraciones y Visión a Largo Plazo

Integraciones con Apps ya en uso

Discord RPC + Webhook

Cambiar estado "Ahora jugando", enviar mensajes a canales definidos, alertar cuando llega un mensaje prioritario. "Elfie, dile a #wolves que llego en 10 min". Discord ya está en LAUNCHERS.

Spotify Web API real

Reemplazar los presets estáticos actuales por la API real: reproducir cualquier playlist/canción por nombre, ver canción actual, control de reproducción. El GOOGLE_CLIENT_ID en config.js ya muestra el patrón OAuth a seguir.

OBS Studio WebSocket

Control de escenas, iniciar/detener grabación. "Elfie, empieza la grabación", "Elfie, cambia a escena de juego". Útil para contenido/streams.

Google Calendar bidireccional

Actualmente PRTS solo escribe a Calendar (recordatorios). Elfie Desktop puede leer los eventos del día en el briefing sin abrir el navegador. Sincronización en ambas vías.

Time Tracker automático

Detectar qué ventana tiene el foco (GetForegroundWindow en Windows) y registrar tiempo activo por app. Elfie analiza: "Pasaste 3 h en VS Code hoy." Sin servicios externos, datos locales.

Screenshot + descripción IA

Elfie captura pantalla y el LLM multimodal (llama3.2-vision vía Ollama) la describe o extrae texto/código. Útil para agregar contexto visual al inbox sin teclear.

Visión a Largo Plazo

Módulo Alumnos (Wolves + LevelUp)

Registro de asistencia, progreso por alumno, calendario de sesiones, notas de clase. Elfie sugiere qué repasar en la próxima sesión basándose en el historial. Se implementa en Fase 5.

Monitor de progreso TecNM

Módulo Escuela: fechas de exámenes, proyectos, calificaciones. Elfie alerta cuando se acerca un parcial y conecta con los Apuntes de esa materia automáticamente.

Fine-tuning con datos propios

Con 6+ meses de datos en Supabase (tarefas, inbox, gym, dieta), hacer fine-tuning de un modelo pequeño (Phi-3.5, Qwen2.5) para que entienda el vocabulario y contexto de Sylft sin prompts largos.

API local de Elfie

Exponer un endpoint REST local (localhost:XXXX) para que scripts, VS Code y otras herramientas le hablen a Elfie directamente sin abrir la app. "curl localhost:7300/elfie?q=modo+estudio"

Resumen de Prioridades

MEJORA / EXPANSIÓN	PRIORIDAD	FASE	JUSTIFICACIÓN
tauri-plugin-global-shortcut	ALTA	Fase 1	Infraestructura básica del asistente de escritorio
Routine Engine Visual	ALTA	Fase 5	Caso de uso diario real, sustituye código hardcodeado
RAG local (LanceDB)	MEDIA	Fase 6	Válido solo para Apuntes, requiere volumen acumulado
Integración Obsidian	CONDICIONAL	Fase 6	Solo si Obsidian ya es parte del flujo diario
Spotify Web API real	MEDIA	Post F5	Mejora sobre presets estáticos actuales
Time Tracker automático	MEDIA	Post F5	Datos de productividad sin servicios externos
Fine-tuning propio	FUTURA	Post F7	Requiere 6+ meses de datos acumulados

